Enhancing Symbolic Execution with Self-Configuring Parameters

Minjong Kim
Sungkyunkwan University
Suwon, Republic of Korea

Sooyoung Cha
Sungkyunkwan University
Suwon, Republic of Korea

ICSE 2026

2026.05.28

백가현

Parameters in Symbolic Execution

- **Symbolic execution**
  - 프로그램의 입력을 symbolic variable로 두고 여러 실행 경로를 탐색하면서 test case를 생성하는 기법.

- 현대 symbolic execution tools 에는 많은 파라미터가 있고 이러한 방대한 파라미터는 사용자로 하여금 사용을 어렵게 만든다.
  - KLEE 는 148개의 parameter 가 있다.

- Symbolic executor 의 많은 외부 파라미터가 branch coverage와 bug finding 등의 성능에 큰 영향을 준다.

Parasuit

- **Parasuit 의 목표**:
  - 각각의 프로그램에 대해 자동으로 symbolic execution parameter configuration을 구성.

구분	SymTuner	ParaSuit
파라미터 선택	사람이 미리 선택	자동 선택
후보값	사람이 정의	프로그램별 자동 구성
튜닝 방식	user-configured	self-configuring
한계	fixed search space	실행 feedback 기반 반복 개선

Existing Parameter Tuning Approaches

- **Manual Tuning Approach**
  - Boolean, integer, string type 파라미터가 섞여 있고 가능한 조합이 매우 많음.

```
$ klee --simplify-sym-indices=true --max-memory=1000
      --max-static-fork-pct=1.0 --search=random-path
              ... (24 more parameters)
      --max-solver-time=30s --sym-args 0 1 10 --sym-files 1 8
```

(a) Parameter values of KLEE used in [66]

- **User-configured Tuning Approach**
  - 사용자가 미리 정한 parameter space 를 입력으로 받고, symbolic execution 을 반복 실행하면서 sampling probability 를 업데이트한다.
  - 그러나, parameter space 자체는 사람이 정해야 한다는 한계가 존재한다.

```
"space": {
   "-simplify-sym-indices": [True, False],
   "-max-memory": [500, 1000, 1500, 2000, 2500],
   "-max-static-fork-pct": [0.25, 0.5, 1, 2, 4],
   "-search": ["random-path", ... , "dfs"],
            ... (13 more spaces)
   "-max-solver-time" : ["10s", "20s", "30s", "40s", "50s"],
   "-sym-arg": [1, 2, 3, 4, 5, 6, 7, 8, 9, 10],
   "-sym-files 1": [4, 8, 12, 16, 20]
}
```

(b) Parameter spaces of KLEE used in [20]

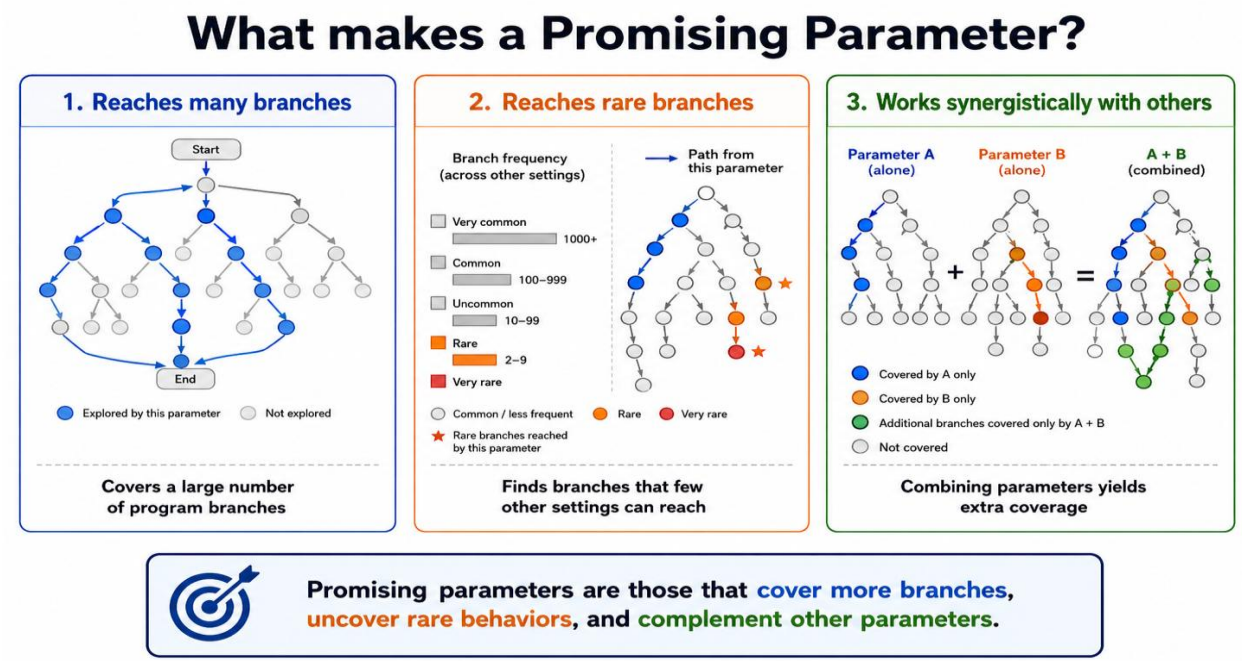
Related Work

- **Enhancing Symbolic Execution**
  - Path explosion을 줄이기 위한 search heuristic 연구
  - Seed selection, state pruning, concolic testing 등으로 탐색 효율 개선
  - 대부분 engine 내부 알고리즘 개선에 초점
- **Tuning External Parameters**
  - KLEE/CREST 같은 도구는 많은 외부 파라미터에 민감
  - SymTuner는 주어진 후보 공간 안에서 값을 적응적으로 선택
  - 하지만 튜닝할 파라미터와 후보 값은 사용자가 미리 정해야 함
- **Search-based Software Testing**
  - Random search, Bayesian optimization, genetic algorithm 등 활용 가능
  - 일반 최적화 기법은 symbolic execution 특화 정보가 부족
  - ParaSuit는 branch feedback을 이용해 프로그램별 설정을 학습

Design

- **Promising parameter**
  - Reach many branches
  - Cover infrequently reached branches
  - Produce synergy when combined with other parameters

- **Score Function**
  - The number of branches covered when adjusting p
  - Its ability to reach branches that are rarely covered by other parameters



PARASUIT

- **Extraction**
  - 심볼릭 실행 도구의 소스코드 AST 및 도움말을 분석하여 모든 가능한 파라미터를 자동 식별
- **Selection**
  - 도달하기 어려운 희귀 브랜치 커버 여부와 파라미터 간의 시너지를 기준으로 점수를 부여
- **Sampling**
  - 프로그램 특성에 맞춰 탐색(Explore) 및 활용(Exploit) 과정을 반복하며 최적 값 도출

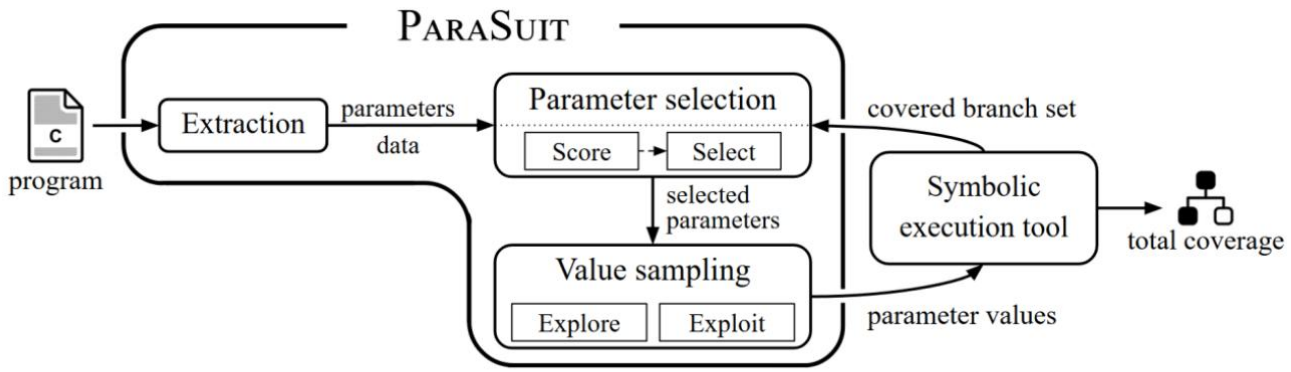


Figure 2: Overview of ParaSuit.

Contributions

- **최초의 완전 자율 튜닝 (Self-configuring)**
  - 사용자 개입 없이, scratch 부터 프로그램 특성에 맞춰 기호 실행 파라미터 최적화

- **맞춤형 탐색 알고리즘 개발**
  - 사전 지식 없이 유효 파라미터를 자동 식별
  - 실행 피드백 기반의 반복적 개선(iterative refinement)으로 최적값 도출

Algorithm 1

- Initialize

	초기화	
D	0	반복 단계에서 축적되는 데이터
BaseD	0	Extraction stage 에서 만든 baseline 데이터
TotalB	0	지금까지 여러 실행에서 나온 branch coverage 결과들의 모음
times	ηtime	Symbolic execution 한 번에 주는 작은 시간 예산

Algorithm 1 PARASUIT

Input: Program (*pgm*), time budget (*time*)
Output: A total set of covered branches (*UnionB*)

1: $\langle D, BaseD, TotalB, time_s \rangle \leftarrow \langle \emptyset, \emptyset, \emptyset, \eta_{\text{time}} \rangle$ ▷ $\eta_{\text{time}} = 120$
2:
3: /* Step 1. Extraction stage */
4: $P \leftarrow$ Initialize() ▷ $(p, v_d, V) \in P$
5: **for** each $(p, v_d, V) \in P$ **do**
6: $PV_d \leftarrow \{(p', v_d) \mid (p', v_d, \_) \in P \wedge (p \neq p')\}$
7: **for** v in $V \setminus \{v_d\}$ **do**
8: $PV \leftarrow PV_d \cup \{(p, v)\}$
9: $B \leftarrow$ EXECUTE$(pgm, time_s, PV)$ ▷ B = covered branches
10: $BaseD \leftarrow BaseD \cup \{(p, v, B)\}$
11: $TotalB \leftarrow TotalB \cup \{B\}$
12:
13: **repeat**
14: /* Step 2. Parameter selection stage */
15: $BaseS \leftarrow \{(p, \text{Score}(B, TotalB)) \mid (p, \_, B) \in BaseD\}$
16: $ParamS \leftarrow \{(p, \text{Score}(B, TotalB)) \mid (p, \_, B) \in D\}$
17: $SelectP \leftarrow$ Select$(ParamS, BaseS)$
18:
19: /* Step 3. Value sampling stage */
20: $PV \leftarrow$ SAMPLE$(SelectP, D)$
21: $B \leftarrow$ EXECUTE$(pgm, time_s, PV)$
22: $TotalB \leftarrow TotalB \cup \{B\}$
23: $D \leftarrow D \cup \{(p, v, B) \mid (p, v) \in PV\}$
24: **until** *time* expires
25: **return** *UnionB* ▷ $UnionB = \bigcup_{B \in TotalB} B$

Algorithm 1

- Initialize
  - D, BaseD, TotalB, times
- **Extraction Stage**
  - 전체 파라미터 후보 추출
  - --help로 parameter 이름 추출

p	파라미터 이름
vd	해당 파라미터의 default value
V	실험해볼 수 있는 possible values

Algorithm 1 PARASUIT

Input: Program (*pgm*), time budget (*time*)
Output: A total set of covered branches (*UnionB*)

1: $\langle D, BaseD, TotalB, time_s \rangle \leftarrow \langle \emptyset, \emptyset, \emptyset, \eta_{\text{time}} \rangle$ ▷ $\eta_{\text{time}} = 120$
2:
3: /* Step 1. Extraction stage */
4: $P \leftarrow$ Initialize() ▷ $(p, v_d, V) \in P$
5: **for** each $(p, v_d, V) \in P$ **do**
6: $PV_d \leftarrow \{(p', v_d) \mid (p', v_d, \_) \in P \wedge (p \neq p')\}$
7: **for** v in $V \setminus \{v_d\}$ **do**
8: $PV \leftarrow PV_d \cup \{(p, v)\}$
9: $B \leftarrow$ EXECUTE$(pgm, time_s, PV)$ ▷ B = covered branches
10: $BaseD \leftarrow BaseD \cup \{(p, v, B)\}$
11: $TotalB \leftarrow TotalB \cup \{B\}$
12:
13: **repeat**
14: /* Step 2. Parameter selection stage */
15: $BaseS \leftarrow \{(p, \text{Score}(B, TotalB)) \mid (p, \_, B) \in BaseD\}$
16: $ParamS \leftarrow \{(p, \text{Score}(B, TotalB)) \mid (p, \_, B) \in D\}$
17: $SelectP \leftarrow \text{Select}(ParamS, BaseS)$
18:
19: /* Step 3. Value sampling stage */
20: $PV \leftarrow$ SAMPLE$(SelectP, D)$
21: $B \leftarrow$ EXECUTE$(pgm, time_s, PV)$
22: $TotalB \leftarrow TotalB \cup \{B\}$
23: $D \leftarrow D \cup \{(p, v, B) \mid (p, v) \in PV\}$
24: **until** *time* expires
25: **return** *UnionB* ▷ $UnionB = \bigcup_{B \in TotalB} B$

Algorithm 1

- Initialize
  - D, BaseD, TotalB, times
- Extraction Stage
  - 전체 파라미터 후보 추출
  - P: 파라미터 이름
  - vd: 그 파라미터의 default value
  - V: 실험해볼 수 있는 possible values
  - **Boolean/string 파라미터**
    - ParaSuit는 source code의 parameter definition block에서 가능한 후보 값을 직접 뽑는다.
  - **Integer/float 파라미터**
    - default value의 0.5배에서 1.5배 사이 값을 후보로 잡는다.

Algorithm 1 PARASUIT

Input: Program (pgm), time budget ($time$)
Output: A total set of covered branches ($UnionB$)

1: $\langle D, BaseD, TotalB, time_s \rangle \leftarrow \langle \emptyset, \emptyset, \emptyset, \eta_{\text{time}} \rangle$ ▷ $\eta_{\text{time}} = 120$
2:
3: /* Step 1. Extraction stage */
4: $P \leftarrow$ Initialize() ▷ $(p, v_d, V) \in P$
5: **for** each $(p, v_d, V) \in P$ **do**
6: $PV_d \leftarrow \{(p', v_d) \mid (p', v_d, \_) \in P \wedge (p \neq p')\}$
7: **for** v in $V \setminus \{v_d\}$ **do**
8: $PV \leftarrow PV_d \cup \{(p, v)\}$
9: $B \leftarrow$ EXECUTE$(pgm, time_s, PV)$ ▷ B = covered branches
10: $BaseD \leftarrow BaseD \cup \{(p, v, B)\}$
11: $TotalB \leftarrow TotalB \cup \{B\}$
12:
13: **repeat**
14: /* Step 2. Parameter selection stage */
15: $BaseS \leftarrow \{(p, \text{Score}(B, TotalB)) \mid (p, \_, B) \in BaseD\}$
16: $ParamS \leftarrow \{(p, \text{Score}(B, TotalB)) \mid (p, \_, B) \in D\}$
17: $SelectP \leftarrow \text{Select}(ParamS, BaseS)$
18:
19: /* Step 3. Value sampling stage */
20: $PV \leftarrow$ SAMPLE$(SelectP, D)$
21: $B \leftarrow$ EXECUTE$(pgm, time_s, PV)$
22: $TotalB \leftarrow TotalB \cup \{B\}$
23: $D \leftarrow D \cup \{(p, v, B) \mid (p, v) \in PV\}$
24: **until** $time$ expires
25: **return** $UnionB$ ▷ $UnionB = \bigcup_{B \in TotalB} B$

Algorithm 1

- Initialize
  - D, BaseD, TotalB, times
- Extraction Stage
  - 전체 파라미터 후보 추출
  - P: 파라미터 이름
  - vd: 그 파라미터의 default value
  - V: 실험해볼 수 있는 possible values
  - Boolean/string 파라미터
  - Integer/float 파라미터
  - **불필요한 parameter filtering**
    - log, version 과 같은 파라미터 제외

Algorithm 1 PARASUIT

Input: Program (*pgm*), time budget (*time*)
Output: A total set of covered branches (*UnionB*)

$\langle D, BaseD, TotalB, time_s \rangle \leftarrow \langle \emptyset, \emptyset, \emptyset, \eta_{\text{time}} \rangle$ ▷ $\eta_{\text{time}} = 120$

/* Step 1. Extraction stage */
$P \leftarrow$ Initialize() ▷ $(p, v_d, V) \in P$
for each $(p, v_d, V) \in P$ **do**
  $PV_d \leftarrow \{(p', v_d) \mid (p', v_d, \_) \in P \wedge (p \neq p')\}$
  for v in $V \setminus \{v_d\}$ **do**
    $PV \leftarrow PV_d \cup \{(p, v)\}$
    $B \leftarrow$ EXECUTE$(pgm, time_s, PV)$ ▷ B = covered branches
    $BaseD \leftarrow BaseD \cup \{(p, v, B)\}$
    $TotalB \leftarrow TotalB \cup \{B\}$

repeat
  /* Step 2. Parameter selection stage */
  $BaseS \leftarrow \{(p, \text{Score}(B, TotalB)) \mid (p, \_, B) \in BaseD\}$
  $ParamS \leftarrow \{(p, \text{Score}(B, TotalB)) \mid (p, \_, B) \in D\}$
  $SelectP \leftarrow \text{Select}(ParamS, BaseS)$

  /* Step 3. Value sampling stage */
  $PV \leftarrow$ SAMPLE$(SelectP, D)$
  $B \leftarrow$ EXECUTE$(pgm, time_s, PV)$
  $TotalB \leftarrow TotalB \cup \{B\}$
  $D \leftarrow D \cup \{(p, v, B) \mid (p, v) \in PV\}$
until *time* expires
return *UnionB* ▷ $UnionB = \bigcup_{B \in TotalB} B$

Algorithm 1

- Initialize
  - D, BaseD, TotalB, times
- Extraction Stage
  - 전체 파라미터 후보 추출
  - P: 파라미터 이름
  - vd: 그 파라미터의 default value
  - V: 실험해볼 수 있는 possible values
  - Boolean/string 파라미터
  - Integer/float 파라미터
  - 불필요한 parameter filtering
  - **파라미터의 단독 영향 평가**
    - 한 번에 하나의 파라미터만 default가 아닌 값으로 바꾸고, 나머지는 모두 default로 둔 뒤 실행.

Algorithm 1 PARASUIT

Input: Program (*pgm*), time budget (*time*)
Output: A total set of covered branches (*UnionB*)

1: $\langle D, BaseD, TotalB, time_s \rangle \leftarrow \langle \emptyset, \emptyset, \emptyset, \eta_{\text{time}} \rangle$ ▷ $\eta_{\text{time}} = 120$
2:
3: /* Step 1. Extraction stage */
4: $P \leftarrow$ Initialize() ▷ $(p, v_d, V) \in P$
5: **for** each $(p, v_d, V) \in P$ **do**
6: $PV_d \leftarrow \{(p', v_d) \mid (p', v_d, \_) \in P \wedge (p \neq p')\}$
7: **for** v in $V \setminus \{v_d\}$ **do**
8: $PV \leftarrow PV_d \cup \{(p, v)\}$
9: $B \leftarrow$ EXECUTE$(pgm, time_s, PV)$ ▷ B = covered branches
10: $BaseD \leftarrow BaseD \cup \{(p, v, B)\}$
11: $TotalB \leftarrow TotalB \cup \{B\}$
12:
13: **repeat**
14: /* Step 2. Parameter selection stage */
15: $BaseS \leftarrow \{(p, \text{Score}(B, TotalB)) \mid (p, \_, B) \in BaseD\}$
16: $ParamS \leftarrow \{(p, \text{Score}(B, TotalB)) \mid (p, \_, B) \in D\}$
17: $SelectP \leftarrow$ Select$(ParamS, BaseS)$
18:
19: /* Step 3. Value sampling stage */
20: $PV \leftarrow$ SAMPLE$(SelectP, D)$
21: $B \leftarrow$ EXECUTE$(pgm, time_s, PV)$
22: $TotalB \leftarrow TotalB \cup \{B\}$
23: $D \leftarrow D \cup \{(p, v, B) \mid (p, v) \in PV\}$
24: **until** *time* expires
25: **return** *UnionB* ▷ $UnionB = \bigcup_{B \in TotalB} B$

Algorithm 1

- Initialize
  - D, BaseD, TotalB, times
- Extraction Stage
- **Selection Stage**
  - 어떤 파라미터를 이번 실행에서 튜닝할지 고르는 단계

```
BaseS  ← { (p, Score(B, TotalB)) | (p, _, B) ∈ BaseD } // 파라미터 하나만 바꿨을 때 얼마나 좋은가

ParamS ← { (p, Score(B, TotalB)) | (p, _, B) ∈ D } // 다른 파라미터들과 함께 썼을 때 얼마나 좋은가
SelectP ← Select(ParamS, BaseS)
```

Algorithm 1 PARASUIT

Input: Program (*pgm*), time budget (*time*)
Output: A total set of covered branches (*UnionB*)

1: $\langle D, BaseD, TotalB, time_s \rangle \leftarrow \langle \emptyset, \emptyset, \emptyset, \eta_{\text{time}} \rangle$ ▷ $\eta_{\text{time}} = 120$
2:
3: /* Step 1. Extraction stage */
4: $P \leftarrow$ Initialize() ▷ $(p, v_d, V) \in P$
5: **for** each $(p, v_d, V) \in P$ **do**
6: $PV_d \leftarrow \{(p', v_d) \mid (p', v_d, \_) \in P \wedge (p \neq p')\}$
7: **for** v in $V \setminus \{v_d\}$ **do**
8: $PV \leftarrow PV_d \cup \{(p, v)\}$
9: $B \leftarrow$ EXECUTE$(pgm, time_s, PV)$ ▷ B = covered branches
10: $BaseD \leftarrow BaseD \cup \{(p, v, B)\}$
11: $TotalB \leftarrow TotalB \cup \{B\}$
12:
13: **repeat**
14: /* Step 2. Parameter selection stage */
15: $BaseS \leftarrow \{(p, \text{Score}(B, TotalB)) \mid (p, \_, B) \in BaseD\}$
16: $ParamS \leftarrow \{(p, \text{Score}(B, TotalB)) \mid (p, \_, B) \in D\}$
17: $SelectP \leftarrow \text{Select}(ParamS, BaseS)$
18:
19: /* Step 3. Value sampling stage */
20: $PV \leftarrow$ SAMPLE$(SelectP, D)$
21: $B \leftarrow$ EXECUTE$(pgm, time_s, PV)$
22: $TotalB \leftarrow TotalB \cup \{B\}$
23: $D \leftarrow D \cup \{(p, v, B) \mid (p, v) \in PV\}$
24: **until** *time* expires
25: **return** *UnionB* ▷ $UnionB = \bigcup_{B \in TotalB} B$

Algorithm 2

- Initialize
  - D, BaseD, TotalB, times
- Extraction Stage
- Selection Stage
- **Sampling Stage**
  - **Explore**: 데이터가 부족하거나 cluster가 불안정할 때 다양한 값을 시도
  - **Exploit**: score가 좋은 cluster를 중심으로 값을 샘플링
  - mean shift clustering 사용
  - 선택된 파라미터 SelectP에 대해 실제 값을 샘플링

Algorithm 2 Value Sampling Stage

Input: Selected parameters (*SelectP*), data (D)
Output: Parameter values (PV)

procedure SAMPLE(*SelectP*, D)
$PV \leftarrow \emptyset$
for each $p \in SelectP$ **do**
  $ValueS \leftarrow \{(v, \text{Score}(B', TotalB)) \mid (p = p') \wedge (p', v', B') \in D\}$
  $C \leftarrow \text{Cluster}(ValueS)$ ▷ $C = \{C_1, C_2, \ldots, C_n\}$; $C_i \subseteq ValueS$
  if $\text{Silhouette}(C) \leq \eta_{\text{clust}}$ **then**
    $PV \leftarrow PV \cup (p, \text{Explore}(ValueS))$ ▷ $\eta_{\text{clust}} = 0.7$
  else
    $PV \leftarrow PV \cup (p, \text{Exploit}(C))$ ▷ $C = \{C_1, C_2, \ldots, C_n\}$
return PV

Algorithm 1

- Initialize
  - D, BaseD, TotalB, times
- Extraction Stage
- Selection Stage
- Sampling Stage
- **Symbolic execution 실행**
  - ParaSuit는 새로 샘플링한 PV로 symbolic execution을 실행

Algorithm 1 PARASUIT

Input: Program (*pgm*), time budget (*time*)
Output: A total set of covered branches (*UnionB*)

1: $\langle D, BaseD, TotalB, time_s \rangle \leftarrow \langle \emptyset, \emptyset, \emptyset, \eta_{\text{time}} \rangle$ ▷ $\eta_{\text{time}} = 120$
2:
3: /* Step 1. Extraction stage */
4: $P \leftarrow$ Initialize() ▷ $(p, v_d, V) \in P$
5: **for** each $(p, v_d, V) \in P$ **do**
6: $PV_d \leftarrow \{(p', v_d) \mid (p', v_d, \_) \in P \wedge (p \neq p')\}$
7: **for** v in $V \setminus \{v_d\}$ **do**
8: $PV \leftarrow PV_d \cup \{(p, v)\}$
9: $B \leftarrow$ EXECUTE$(pgm, time_s, PV)$ ▷ B = covered branches
10: $BaseD \leftarrow BaseD \cup \{(p, v, B)\}$
11: $TotalB \leftarrow TotalB \cup \{B\}$
12:
13: **repeat**
14: /* Step 2. Parameter selection stage */
15: $BaseS \leftarrow \{(p, \text{Score}(B, TotalB)) \mid (p, \_, B) \in BaseD\}$
16: $ParamS \leftarrow \{(p, \text{Score}(B, TotalB)) \mid (p, \_, B) \in D\}$
17: $SelectP \leftarrow \text{Select}(ParamS, BaseS)$
18:
19: /* Step 3. Value sampling stage */
20: $PV \leftarrow$ SAMPLE$(SelectP, D)$
21: $B \leftarrow$ EXECUTE$(pgm, time_s, PV)$
22: $TotalB \leftarrow TotalB \cup \{B\}$
23: $D \leftarrow D \cup \{(p, v, B) \mid (p, v) \in PV\}$
24: **until** *time* expires
25: **return** *UnionB* ▷ $UnionB = \bigcup_{B \in TotalB} B$

Algorithm 1

- Initialize
  - D, BaseD, TotalB, times
- Extraction Stage
- Selection Stage
- Sampling Stage
- Symbolic execution 실행
- **데이터 갱신**

Algorithm 1 PARASUIT

Input: Program (*pgm*), time budget (*time*)
Output: A total set of covered branches (*UnionB*)

1: $\langle D, BaseD, TotalB, time_s \rangle \leftarrow \langle \emptyset, \emptyset, \emptyset, \eta_{\text{time}} \rangle$ ▷ $\eta_{\text{time}} = 120$
2:
3: /* Step 1. Extraction stage */
4: $P \leftarrow$ Initialize() ▷ $(p, v_d, V) \in P$
5: **for** each $(p, v_d, V) \in P$ **do**
6: $PV_d \leftarrow \{(p', v_d) \mid (p', v_d, \_) \in P \wedge (p \neq p')\}$
7: **for** v in $V \setminus \{v_d\}$ **do**
8: $PV \leftarrow PV_d \cup \{(p, v)\}$
9: $B \leftarrow$ EXECUTE$(pgm, time_s, PV)$ ▷ B = covered branches
10: $BaseD \leftarrow BaseD \cup \{(p, v, B)\}$
11: $TotalB \leftarrow TotalB \cup \{B\}$
12:
13: **repeat**
14: /* Step 2. Parameter selection stage */
15: $BaseS \leftarrow \{(p, \text{Score}(B, TotalB)) \mid (p, \_, B) \in BaseD\}$
16: $ParamS \leftarrow \{(p, \text{Score}(B, TotalB)) \mid (p, \_, B) \in D\}$
17: $SelectP \leftarrow \text{Select}(ParamS, BaseS)$
18:
19: /* Step 3. Value sampling stage */
20: $PV \leftarrow$ SAMPLE$(SelectP, D)$
21: $B \leftarrow$ EXECUTE$(pgm, time_s, PV)$
22: $TotalB \leftarrow TotalB \cup \{B\}$
23: $D \leftarrow D \cup \{(p, v, B) \mid (p, v) \in PV\}$
24: **until** *time* expires
25: **return** *UnionB* ▷ $UnionB = \bigcup_{B \in TotalB} B$

Algorithm 1

- Initialize
  - D, BaseD, TotalB, times
- Extraction Stage
- Selection Stage
- Sampling Stage
- Symbolic execution 실행
- 데이터 갱신
- **UnionB 반환**
  - 모든 짧은 symbolic execution 실행에서 커버된 branch들을 합친 최종 branch coverage

Algorithm 1 PARASUIT

Input: Program (*pgm*), time budget (*time*)
Output: A total set of covered branches (*UnionB*)

1: $\langle D, BaseD, TotalB, time_s \rangle \leftarrow \langle \emptyset, \emptyset, \emptyset, \eta_{\text{time}} \rangle$ ▷ $\eta_{\text{time}} = 120$
2:
3: /* Step 1. Extraction stage */
4: $P \leftarrow$ Initialize() ▷ $(p, v_d, V) \in P$
5: **for** each $(p, v_d, V) \in P$ **do**
6: $PV_d \leftarrow \{(p', v_d) \mid (p', v_d, \_) \in P \wedge (p \neq p')\}$
7: **for** v in $V \setminus \{v_d\}$ **do**
8: $PV \leftarrow PV_d \cup \{(p, v)\}$
9: $B \leftarrow$ EXECUTE$(pgm, time_s, PV)$ ▷ B = covered branches
10: $BaseD \leftarrow BaseD \cup \{(p, v, B)\}$
11: $TotalB \leftarrow TotalB \cup \{B\}$
12:
13: **repeat**
14: /* Step 2. Parameter selection stage */
15: $BaseS \leftarrow \{(p, \text{Score}(B, TotalB)) \mid (p, \_, B) \in BaseD\}$
16: $ParamS \leftarrow \{(p, \text{Score}(B, TotalB)) \mid (p, \_, B) \in D\}$
17: $SelectP \leftarrow \text{Select}(ParamS, BaseS)$
18:
19: /* Step 3. Value sampling stage */
20: $PV \leftarrow$ SAMPLE$(SelectP, D)$
21: $B \leftarrow$ EXECUTE$(pgm, time_s, PV)$
22: $TotalB \leftarrow TotalB \cup \{B\}$
23: $D \leftarrow D \cup \{(p, v, B) \mid (p, v) \in PV\}$
24: **until** *time* expires
25: **return** *UnionB* ▷ $UnionB = \bigcup_{B \in TotalB} B$

Experiments

Experiments

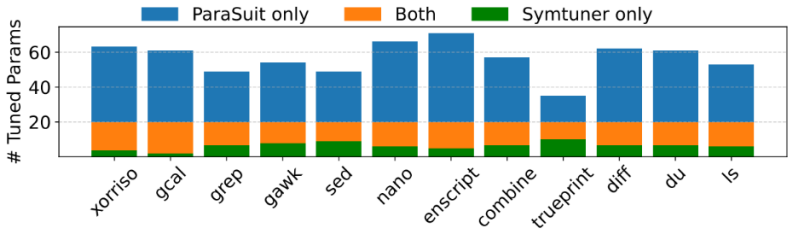
- Settings: 4 baselines
  - $KLEE_{fix}$, $Symtuner$, $Symtuner_{AIIP}$, $Randsuit$
- Benchmark
- Tool: KLEE-2.1
- Benchmarks: 12 GNU C programs
- Budget: 24 hours
- Repetitions: 5 trials
- Metric: branch coverage, unique bugs
- Coverage measurement: gcov

Program	LoC	Branches
xorriso-1.5.2	161K	49,162
gcal-4.1	89K	15,799
grep-3.4	82K	8,225
gawk-5.1.0	77K	10,720
sed-4.8	66K	6,798
nano-4.9	54K	10,436
enscript-1.6.6	49K	3,796
combine-0.4.0	32K	2,357
trueprint-5.4	12K	2,518
diff-3.7	9K	7,612
du-8.32	8K	6,653
ls-8.32	5K	3,776

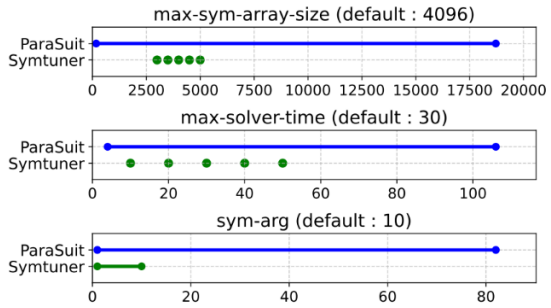
Benchmark

Branch Coverage

- $KLEE_{fix}$ 보다 97.4%
- $Symtuner$ 보다 25.8%
- $Symtuner_{AIIP}$ 보다 95.6%
- $Randsuit$ 보다 46.8%



(a) The difference in the parameter selection.



(b) The difference in the value sampling.

Table 2: Branch coverage achieved by ParaSuit and four baselines on 12 benchmarks.

Benchmark	ParaSuit	Symtuner	$KLEE_{fix}$	RandSuit	$Symtuner_{AIIP}$
xorriso-1.5.2	**6,109.4**	2,956.8	2,014.8	2,272.8	2,117.6
gcal-4.1	**4,766.0**	3,907.0	2,003.2	3,234.6	1,825.0
grep-3.4	**3,613.8**	3,280.2	2,208.4	3,344.2	2,017.8
gawk-5.1.0	**4,393.2**	3,747.0	2,841.6	2,893.0	3,261.8
sed-4.8	**2,462.6**	2,357.8	1,097.0	1,840.8	1,267.6
nano-4.9	**1,909.6**	1,721.0	1,457.0	1,662.2	1,289.0
enscript-1.6.6	**776.4**	734.2	580.8	688.8	448.6
combine-0.4.0	**1,004.6**	952.4	644.8	835.6	463.4
trueprint-5.4	**1,300.2**	1,019.2	579.0	881.6	571.2
diff-3.7	**2,139.0**	1,762.2	538.4	1,462.4	1,034.2
du-8.32	**1,238.6**	1,190.6	829.2	1,187.8	814.0
ls-8.32	**2,165.4**	1,712.0	1,355.2	1,410.4	1,185.8
total	**31,878.8**	25,340.4	16,149.4	21,714.2	16,296.0

Bug-finding ability

- A bug-triggering test-case was considered unique if its error location reported by KLEE differed from others
- Parasuit succeeded in detecting 11 unique bugs across three of the 12 programs: gcal, gawk, and combine. (SYMTUNER 보다 더 좋은 성능.)

Table 3: Number of bugs detected by PARASUIT and baselines.

Benchmark	PARASUIT	SYMTUNER	$KLEE_{fix}$	RANDSUIT	$SYMTUNER_{AllP}$
gcal	**7**	4	4	2	1
gawk	**1**	**1**	-	**1**	-
combine	**3**	2	1	1	1
total	**11**	7	5	4	3

Efficacy of two stages

- Parameter selection stage

  (xorriso, gcal, grep) with (gcal, gawk, combine)

  RandP, AllP, SymP보다 각각 60.2%, 89.9%, 34.2% 더 높은 coverage

- Value sampling algorithm
  - CMA-ES보다 **11.0%**, RandV보다 **21.5%**, SymV보다 49.0% 더 높은 coverage

$$BaseS \leftarrow \{(p, \text{Score}(B, TotalB)) \mid (p, \_, B) \in BaseD\}$$
$$ParamS \leftarrow \{(p, \text{Score}(B, TotalB)) \mid (p, \_, B) \in D\}$$

Table 4: Effectiveness of our parameter selection stage compared with naive selection approaches.

Benchmark	ParaSuit	RandP	SymP	AllP
xorriso	**6,109(0)**	2,615(0)	3,149(0)	2,426(0)
gcal	**4,766(7)**	3,578(2)	4,160(3)	3,105(2)
grep	**3,614(0)**	2,852(0)	3,404(0)	1,749(0)
gawk	**4,393(1)**	2,551(0)	3,336(1)	2,521(1)
combine	**1,005(3)**	861(1)	953(2)	577(1)
total	**19,887(11)**	12,457(3)	15,002(6)	10,378(4)

Table 5: The efficiency of the value sampling algorithm compared with five sampling methods.

Benchmark	ParaSuit	Bayesian	Genetic	CMA-ES	RandV	SymV
xorriso	**6,109(0)**	4,417(0)	4,383(0)	5,361(0)	3,759(0)	2,745(0)
gcal	**4,766(7)**	4,617(5)	4,552(5)	4,412(5)	4,421(4)	3,642(2)
grep	**3,614(0)**	3,559(0)	3,349(0)	3,455(0)	3,407(0)	3,414(0)
gawk	**4,393(1)**	4,116(1)	3,902(0)	3,801(1)	3,920(0)	2,694(0)
combine	**1,005(3)**	883(1)	922(1)	892(2)	868(1)	856(1)
total	**19,887(11)**	17,592(7)	17,108(6)	17,921(8)	16,375(5)	13,351(3)

Analysis of self-configuration

- Impact of Selected Parameters
- Tendency of Selections and Sampled Values
- Efficiency of Self-configured Parameter Sets

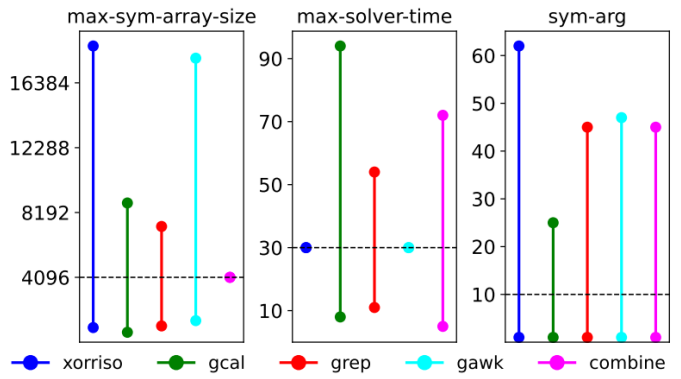


Figure 5: Integer parameter values sampled by ParaSuit.

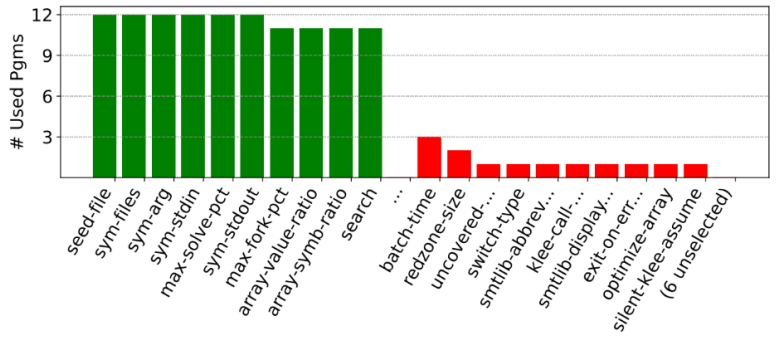


Figure 4: The count of benchmarks where each parameter was chosen at least once.

Generality

- 다른 symbolic/concolic testing 도구에도 적용 가능한지
- ParaSuit + Chameleon은 Chameleon 단독보다 평균 3.5배 더 많은 unique branches를 커버
- 총 6개 distinct bugs 발견
- vim과 sed에서 추가 bug 발견

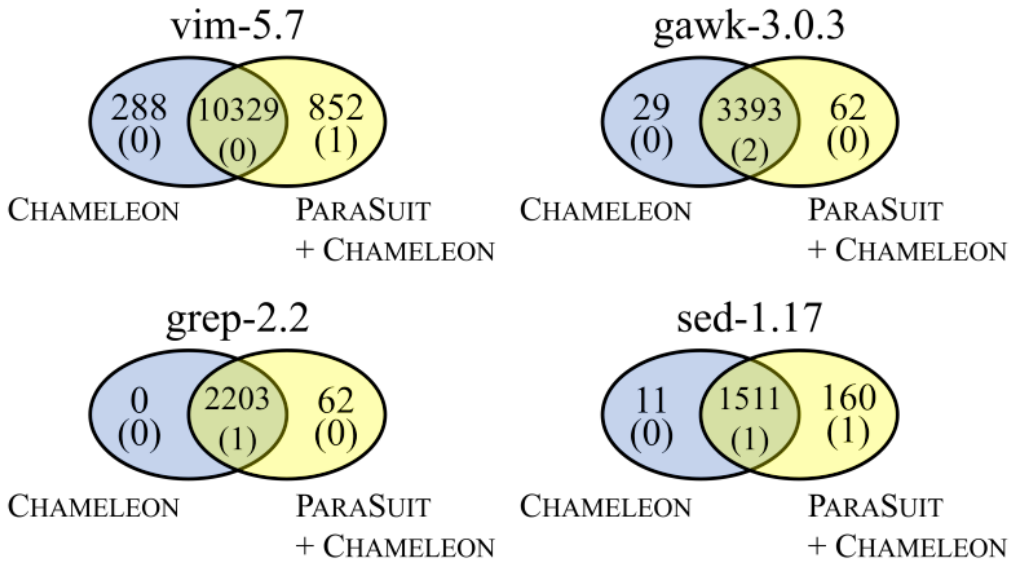


Figure 6: Efficacy of parameter tuning for concolic testing.

Threats to Validity

- ParaSuit는 두 개의 hyper-parameter를 사용
- benchmark가 12개 C 프로그램으로 제한

Conclusion

- 사용자 개입 없이 symbolic execution 파라미터를 처음부터 최적화하는 최초의 Self-configuring 기술 제안.
- 타겟 프로그램에 대한 symbolic execution feedback을 기반으로 파라미터를 자동 선택·샘플링.
- 다양한 벤치마크에서 branch coverage 대폭 향상 및 기존에 놓쳤던 고유 버그들을 발견.

Thank You